



PROJETO PS 2026.1



Gerente responsável

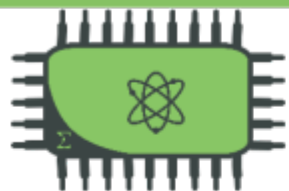
Vitor Teixeira

Data de início

17/04/2026

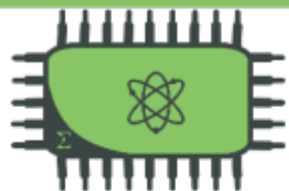
Data de entrega

02/05/2026



Sumário

1	Introdução	2
2	Análise de Requisitos	3
2.1	Convenções, termos e abreviações	3
2.1.1	Identificação dos Requisitos	3
2.1.2	Propriedades dos requisitos	3
2.2	Requisitos Funcionais	4
2.3	Requisitos Não Funcionais	6
3	Fluxograma do Software	8
4	Protótipo de Interface	9
5	Diagrama de Blocos	10
6	Modelagem do Circuito	11
7	Descrição de Hardware	12
7.1	Arquitetura Geral	12
7.2	Componentes e Justificativas Técnicas	12
7.3	Dimensionamento e Alimentação	13
7.4	Estrutura Física e Montagem	13
7.5	Mapa de Conexões (Pinagem Validada)	13
8	Descrição de Alto nível	14
8.1	Subsistema de Alimentação e Regulação	14
8.2	Subsistema de Sensoriamento e Armazenamento	14
8.3	Subsistema de Interface e Atuação	14
8.4	Subsistema de Comunicação e Arquitetura de Software	15
9	Descrição de Firmware	16
9.1	Sequência de Boot e Inicialização	16
9.2	Ciclo de Execução e Máquina de Estados	16
9.3	Gerenciamento de Arquivos no microSD	16
9.4	Protocolo de Comunicação Web	17
10	Bill of Materials	18



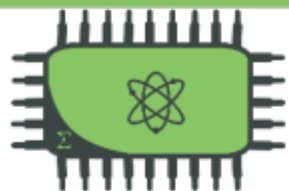
1 Introdução

O projeto tem como objetivo o desenvolvimento de um sistema de monitoramento capaz de coletar dados de temperatura e umidade do ambiente e disponibilizá-los em tempo real por meio de uma interface web.

O sistema será composto por um sensor de temperatura e umidade, responsável pela aquisição dos dados ambientais, e por um microcontrolador, encarregado de processar essas informações, realizar a comunicação com a interface web e controlar os elementos de saída do sistema, como os LEDs indicadores de estado. A escolha desses componentes será fundamentada em critérios técnicos relevantes, como faixa de medição, precisão, consumo de energia, facilidade de integração e capacidade de comunicação em rede.

Além da aquisição e exibição dos dados, o sistema deverá permitir a interação com o usuário por meio de uma interface web, na qual será possível visualizar as medições em tempo real e alternar entre diferentes modos de operação. Os modos previstos são: desligado, no qual o sistema permanece inativo; automático, em que a coleta e exibição dos dados ocorrem de forma contínua; e ligado, no qual o sistema aguarda ações do usuário. A mudança de estado deverá ser refletida visualmente através de LEDs, proporcionando uma indicação clara e imediata do modo de operação atual.

O desenvolvimento do projeto envolve a integração entre hardware e software, implementando a lógica de funcionamento do sistema e da interface web. Dessa forma, busca-se construir uma solução funcional, organizada e alinhada aos requisitos propostos.



2 Análise de Requisitos

A análise de requisitos é o levantamento de todas as propriedades, características e necessidades do sistema para que esse seja capaz de executar, com a qualidade desejada, as funcionalidades previstas. Essa seção serve de base tanto para o cliente, guiando as expectativas quanto ao projeto a partir da definição sólida de quais serão suas especificações, como para os projetistas, que, ao desenvolverem o sistema de acordo com os requisitos levantados, estarão seguros quanto à sua efetividade para o problema.

Os requisitos funcionais, dispostos na seção 2.2, são propriedades do sistema responsáveis por explicitar suas funcionalidades e como estas atendem às solicitações do projeto. A partir da leitura dos requisitos funcionais, tem-se noção do que o sistema deve ser capaz de executar e quais são as necessidades que ele busca suprir.

Os requisitos não-funcionais, dispostos na seção 2.3, são propriedades do sistema que mostram especificações e características relacionadas ao modo como os requisitos funcionais são operados. A partir da leitura dos requisitos não-funcionais, tem-se noção de informações técnicas e atributos de qualidade de execução das funcionalidades do sistema.

2.1 Convenções, termos e abreviações

2.1.1 Identificação dos Requisitos

Por convenção, a referência a requisitos é feita através do nome da subseção na qual eles estão descritos, seguidos do identificador do requisito, de acordo com o seguinte modelo:

[nome da subseção/identificador do requisito]

Os requisitos serão identificados com um identificador único, com numeração iniciada em [RF01] para um requisito funcional e [NF01] para um requisito não-funcional.

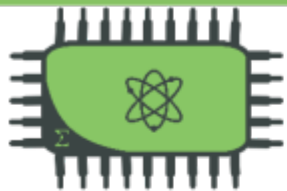
2.1.2 Propriedades dos requisitos

Para estabelecer a prioridade dos requisitos, nas seções 2.2 e 2.3, foram adotadas as denominações “essencial”, “importante” e “desejável”, tal que:

Essencial é o requisito sem o qual o sistema não entra em funcionamento. Requisitos essenciais são aqueles imprescindíveis, que devem ser implementados impreterivelmente.

Importante é o requisito sem o qual o sistema entra em funcionamento, mas de forma não satisfatória. Requisitos importantes devem ser implementados, mas, se não forem, o sistema poderá ser implementado e usado normalmente.

Desejável é o requisito que não compromete as funcionalidades básicas do sistema, isto é, o sistema pode funcionar de forma satisfatória sem ele. Requisitos desejáveis podem ser deixados para versões posteriores do sistema, caso não haja tempo hábil para implementação dos mesmos nesta etapa.



2.2 Requisitos Funcionais

[RF01] [Aquisição de Dados Ambientais]

O sistema deve coletar dados de temperatura e umidade do ambiente por meio de um sensor apropriado.

Prioridade: Essencial.

[RF02] [Disponibilização de Dados em Interface Web]

O sistema deve disponibilizar os dados coletados em uma interface web acessível ao usuário.

Prioridade: Essencial.

[RF03] [Atualização em Tempo Real]

O sistema deve atualizar os dados de temperatura e umidade em tempo real na interface web.

Prioridade: Essencial.

[RF04] [Controle de Modos de Operação]

O sistema deve permitir a alternância entre os modos de operação (Desligado, Automático e Ligado) através da interface web.

Prioridade: Essencial.

[RF05] [Indicação Visual de Estado]

O sistema deve indicar visualmente o modo de operação atual por meio de LEDs no hardware.

Prioridade: Essencial.

[RF06] [Sincronização Interface-Hardware]

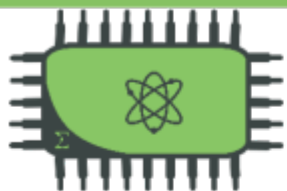
O sistema deve sincronizar o estado dos LEDs com o modo selecionado na interface web.

Prioridade: Essencial.

[RF07] [Operação Automática Contínua]

No modo automático, o sistema deve realizar a coleta e atualização contínua dos dados.

Prioridade: Essencial.



[RF08] [Operação Ativa no Modo Ligado]

No modo ligado, o sistema deve permanecer ativo e apto a responder às ações do usuário.

Prioridade: Essencial.

[RF09] [Desativação do Sistema]

No modo desligado, o sistema deve interromper a coleta de dados e ativar o LED de desligado.

Prioridade: Essencial.

[RF10] [Atualização Dinâmica da Interface]

O sistema deve permitir a interface web atualizada sem necessidade de recarregamento manual da página.

Prioridade: Essencial.

[RF11] [Requisição Manual de Leituras]

No modo ligado, a interface deve disponibilizar botões para requisição manual de leitura de temperatura, umidade ou ambas.

Prioridade: Essencial.

[RF12] [Registro de Histórico de Dados]

O sistema deve armazenar um histórico das leituras com data e hora, exibindo-o na interface web.

Prioridade: Importante.

[RF13] [Persistência de Dados em microSD]

O sistema deve gravar cada leitura em um arquivo .txt no cartão microSD, no formato CSV (data, hora, temperatura, umidade, modo).

Prioridade: Importante.

[RF14] [Configuração de Limites e Alertas]

O sistema deve permitir configurar limites mínimos e máximos e exibir alerta visual quando ultrapassados.

Prioridade: Importante.

[RF15] [Persistência de Estado após Reinicialização]

O sistema deve preservar o último modo de operação após reinicialização da ESP32.

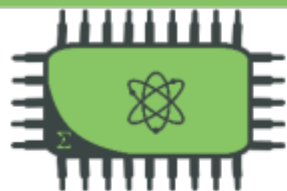
Prioridade: Desejável.

Brasília, 2 de maio de 2026.

ELETRONJUN – UNIVERSIDADE DE BRASÍLIA
EMPRESA JÚNIOR DE ENGENHARIA ELETRÔNICA

Área Especial de Indústria – Projeção A | 72444-240 | Gama/DF

www.eletronjun.com.br | contato@eletronjun.com.br



[RF16] [Configuração do Intervalo de Coleta]

O sistema deve permitir configurar o intervalo entre leituras automáticas.

Prioridade: Desejável.

[RF17] [Notificação por E-mail]

O sistema poderá enviar notificações por e-mail quando limites forem ultrapassados.

Prioridade Desejável.

[RF18] [Disponibilização em Aplicativo Mobile]

O sistema poderá ser disponibilizado como aplicativo mobile utilizando React Native.

Prioridade: Desejável.

[RF19] [Suporte a Múltiplos Dispositivos]

O sistema poderá suportar múltiplos dispositivos ESP32 simultaneamente.

Prioridade: Desejável.

2.3 Requisitos Não Funcionais

[NF01] [Baixa Latência de Atualização]

O sistema deve apresentar baixa latência na atualização dos dados.

Prioridade: Essencial.

[NF02] [Conectividade Wi-Fi]

O microcontrolador deve possuir comunicação via rede Wi-Fi.

Prioridade: Essencial.

[NF03] [Estabilidade Operacional]

O sistema deve operar de forma estável durante sua execução contínua.

Prioridade: Essencial.

[NF04] [Precisão das Medições]

Os dados exibidos devem respeitar a precisão e a faixa de medição do sensor utilizado.

Prioridade: Essencial.

[NF05] [Compatibilidade Hardware-Software]

O sistema deve garantir a correta integração entre sensor, microcontrolador e interface web.

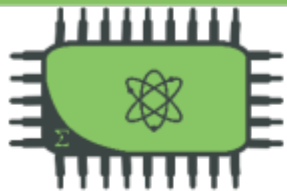
Prioridade: Essencial.

Brasília, 2 de maio de 2026.

ELETRONJUN – UNIVERSIDADE DE BRASÍLIA
EMPRESA JÚNIOR DE ENGENHARIA ELETRÔNICA

Área Especial de Indústria – Projeção A| 72444-240 | Gama/DF

www.eletronjun.com.br | contato@eletronjun.com.br



[NF06] [Interação em Modo Ligado]

A interface deve permitir requisição de temperatura e umidade no modo ligado.
Prioridade: Funcional.

[NF07] [Capacidade de Armazenamento]

O cartão microSD deve comportar pelo menos 6 meses de leituras com intervalo de 1 minuto.
Prioridade: Importante.

[NF08] [Tempo de Resposta de Alertas]

O alerta visual deve ser exibido em no máximo 5 segundos após a leitura crítica.
Prioridade: Importante.

[NF09] [Integridade dos Dados]

A gravação no microSD deve evitar corrupção de dados em caso de falhas.
Prioridade: Importante.

[NF10] [Suporte a Múltiplos Acessos]

O sistema deve suportar pelo menos 4 acessos simultâneos sem falhas.
Prioridade: Desejável.

[NF11] [Recuperação Automática de Falhas]

O sistema poderá implementar watchdog timer para reinicialização automática.
Prioridade: Desejável.

[NF12] [Persistência de Configurações]

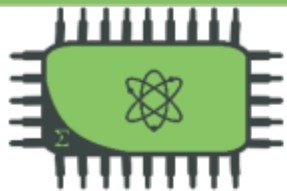
As configurações do usuário poderão ser salvas na memória flash (LittleFS).
Prioridade: Desejável.

[NF13] [Acesso Remoto]

O sistema poderá ser acessado fora da rede local via internet.
Prioridade: Desejável.

[NF14] [Compatibilidade Mobile]

O aplicativo mobile poderá funcionar em Android e iOS com uma única base de código.
Prioridade: Desejável.



3 Fluxograma do Software

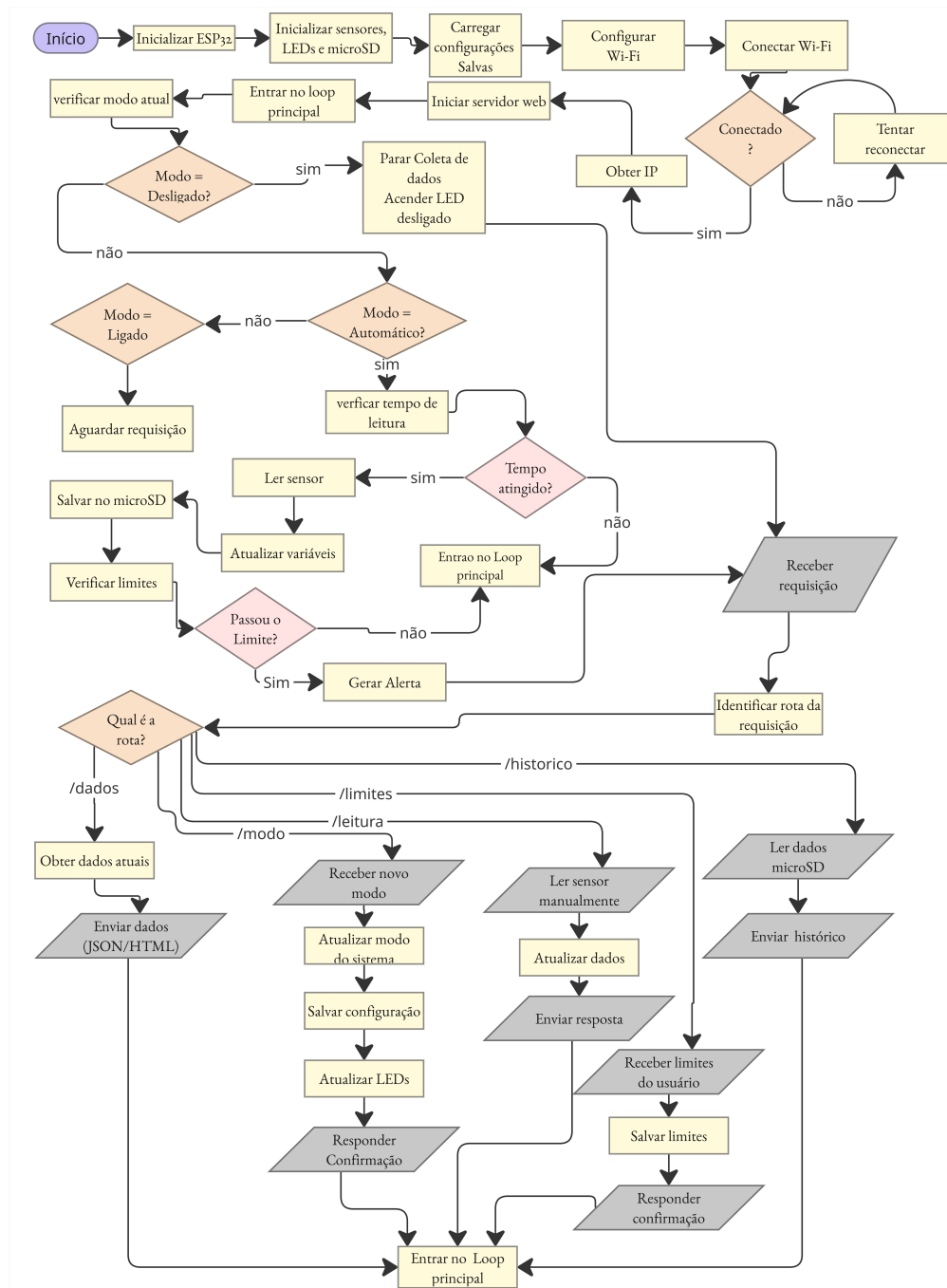


Figura 2: Fluxograma de operação do software no ESP32

Com o intuito de nortear a execução das tarefas e garantir a eficiência dos processos, desenvolveu-se o fluxograma de software apresentado acima (Figura 2). O diagrama detalha as etapas lógicas e as tomadas de decisão que compõem a arquitetura do sistema, demonstrando como ele alterna entre o modo automático de leitura e a escuta ativa de requisições HTTP via Wi-Fi.

4 Protótipo de Interface

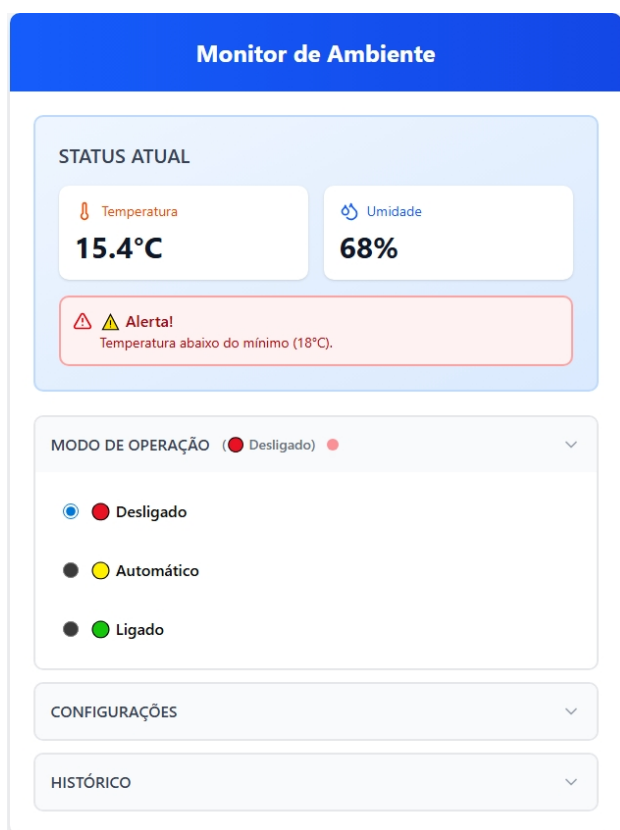
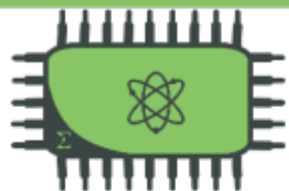


Figura 3: Protótipo de Interface

A Figura 3 apresenta o protótipo da interface web do sistema, que será implementado no ESP32 para monitoramento de temperatura e umidade do ambiente. A interface exibe os valores em tempo real e permite ao usuário controlar o modo de operação do sistema. Além disso, inclui áreas para configurações e visualização de histórico, facilitando o acompanhamento e gerenciamento dos dados coletados.



5 Diagrama de Blocos

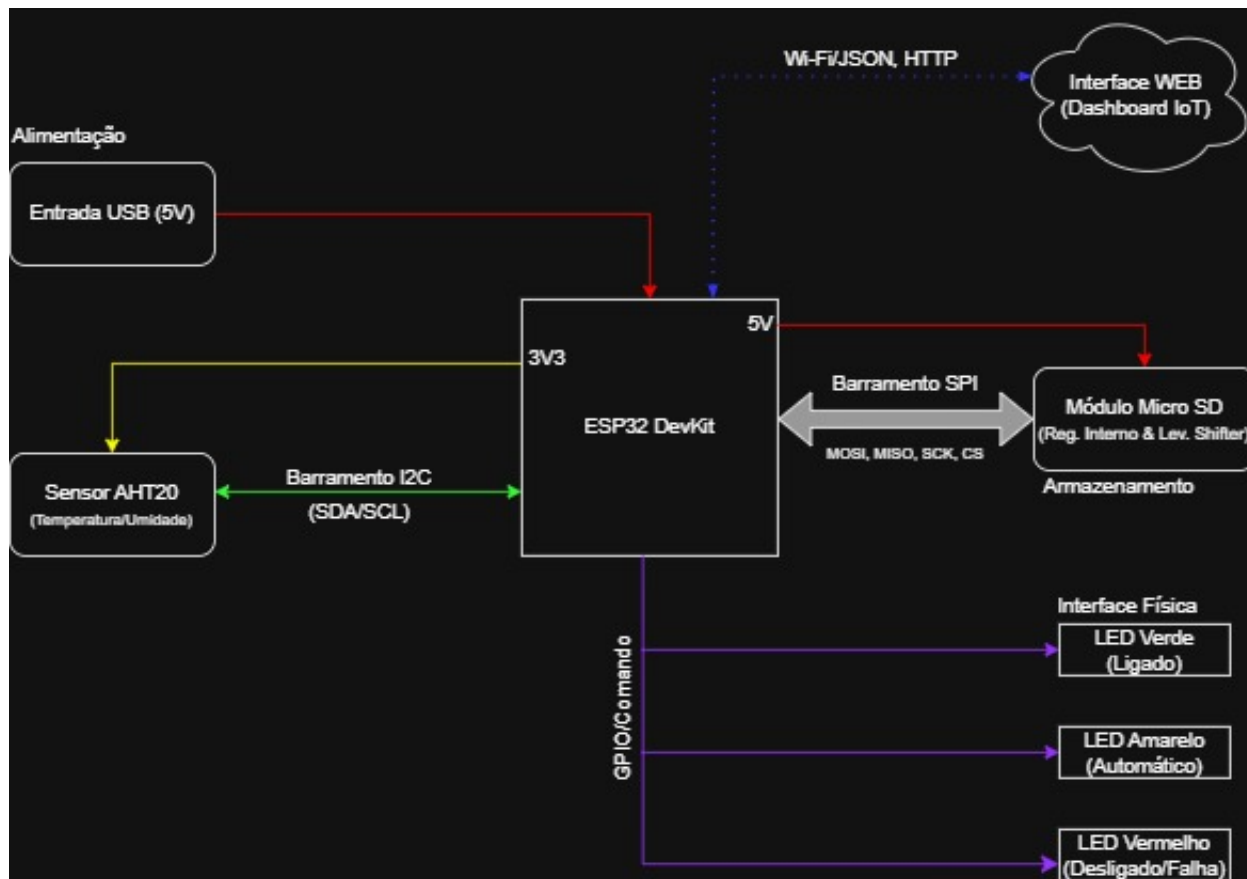
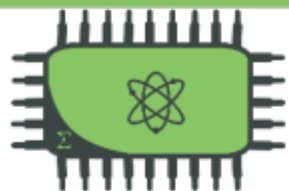


Figura 4: Diagrama de Blocos do Sistema de Monitoramento de temperatura e umidade em ESP32 DevKit

Descrição do Diagrama

O diagrama ilustra a arquitetura completa do sistema de monitoramento de temperatura e umidade do ambiente. O núcleo é o ESP32 DevKit, que recebe alimentação de 5V via USB e distribui internamente a referência de 3,3V para os periféricos. O Sensor AHT20 (temperatura e umidade) comunica-se com o ESP32 pelo barramento I²C (SDA/SCL). Os dados são armazenados localmente em um Módulo Micro SD via barramento SPI (MOSI, MISO, SCK, CS), que possui um level shifter integrado para adequação dos níveis de tensão. O ESP32 também publica as leituras em um Dashboard IoT acessível remotamente por Wi-Fi, utilizando protocolo HTTP com payload em JSON. Por fim, três LEDs controlados por GPIO indicam o estado operacional: verde (ligado), amarelo (automático) e vermelho (desligado/falha).



6 Modelagem do Circuito

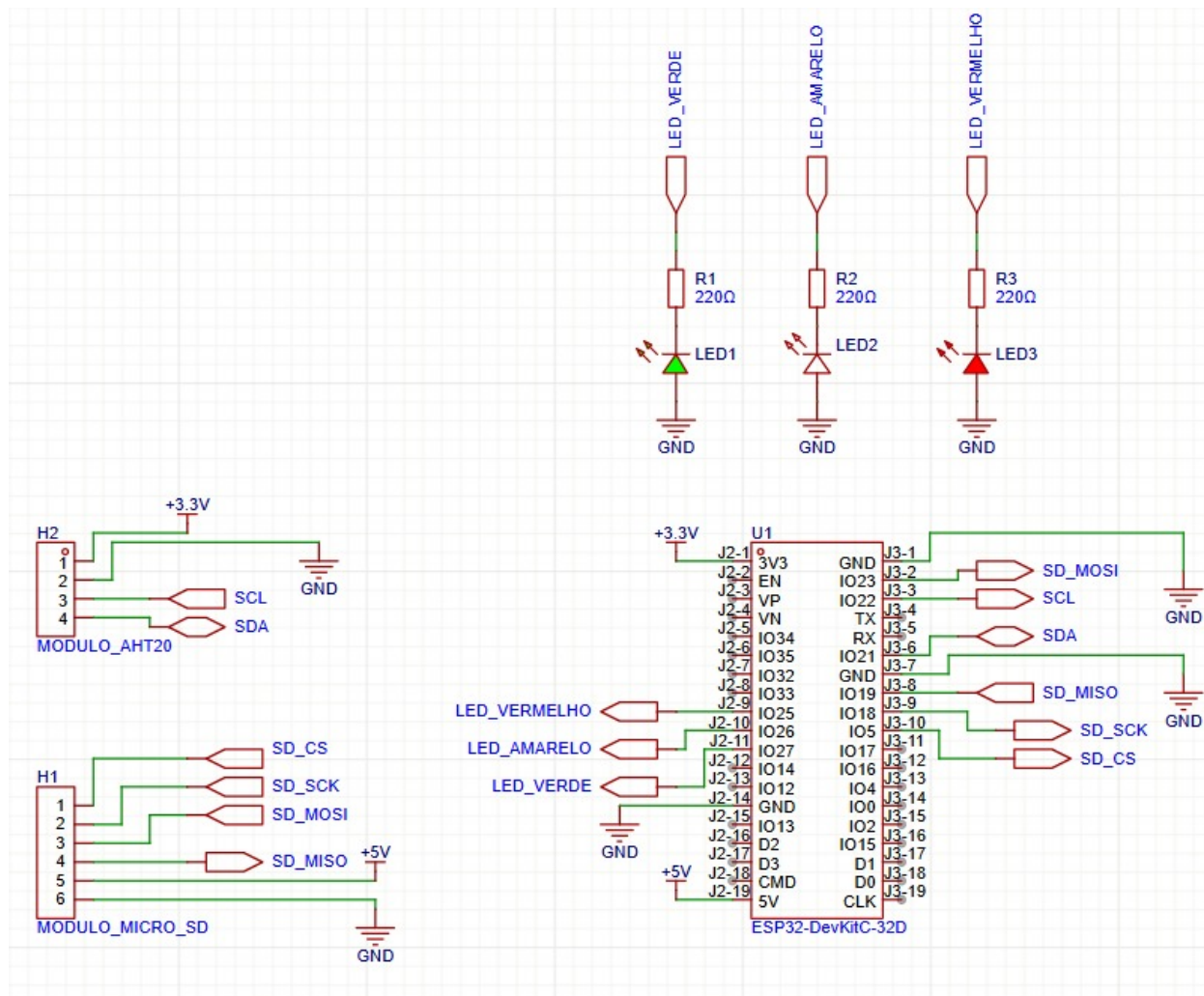
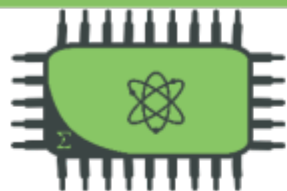


Figura 5: Esquemático de Hardware

O diagrama elétrico detalha as conexões físicas entre o microcontrolador ESP32 e os periféricos do sistema. Nele, destaca-se a integração do sensor AHT20 via barramento I2C e a interface do módulo microSD via protocolo SPI. Além disso, o esquema apresenta o circuito de sinalização visual, composto por três LEDs (Verde, Amarelo e Vermelho) equipados com resistores de 220 Ω para garantir a proteção das portas lógicas do chip



7 Descrição de Hardware

Nesta seção é mostrada a Descrição do Hardware, que são listados os principais componentes que compõem as principais soluções levantadas para o desenvolvimento do projeto. Eles são selecionados com base nas funcionalidades descritas e acordadas com o cliente na etapa de definição de requisitos.

7.1 Arquitetura Geral

O hardware do projeto foi projetado para operar de forma autônoma e centralizada. O sistema baseia-se em um microcontrolador com conectividade Wi-Fi nativa, que atua simultaneamente como gerenciador de aquisição de dados ambientais, controlador de atuadores visuais (LEDs) e hospedeiro do servidor web local. A comunicação entre os módulos periféricos foi distribuída utilizando barramentos padronizados (I2C e SPI) para garantir estabilidade e economia de portas lógicas (GPIOs).

7.2 Componentes e Justificativas Técnicas

- **Microcontrolador: ESP32 DevKitC V4 (WROOM-32D)**

- *Especificações:* Processador Dual Core de 32-bits, Wi-Fi 802.11 b/g/n, operação em 3,3V.
- *Justificativa:* Escolhido por sua capacidade de processamento multitarefa. O uso de núcleos separados permite manter o servidor web local operando de forma estável, sem interromper as rotinas de leitura de sensores e gravação de logs.

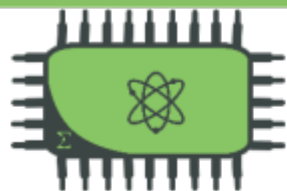
- **Sensor de Temperatura e Umidade: Módulo AHT20**

- *Especificações:* Protocolo I2C, precisão de ± 0.3 °C (Temp) e $\pm 2\%$ (Umid), calibração interna de fábrica.
- *Justificativa:* Superior em estabilidade aos sensores da família DHT. O uso do barramento I2C minimiza o uso de pinos do microcontrolador e evita erros de temporização comuns em protocolos *single-wire*.

- **Módulo de Armazenamento (Log Local): Leitor Micro SD Padrão**

- *Especificações:* Comunicação SPI, regulador de tensão integrado (5V para 3,3V) e chip *Level Shifter*.
- *Justificativa:* O modelo padrão protege o cartão SD e o ESP32 contra picos de tensão. É responsável por gravar as leituras em formato `.csv/.txt`, garantindo a persistência dos dados em caso de falha de conexão.

- **Interface Visual: LEDs**



- *Justificativa:* Indicam os estados de operação (Ligado, Automático, Desligado). Foram dimensionados em série com resistores de $220\ \Omega$, mantendo o consumo individual em cerca de 6mA e protegendo as portas lógicas do ESP32 contra sobrecorrente.

7.3 Dimensionamento e Alimentação

O circuito foi projetado para ser alimentado diretamente pela porta USB de um notebook (5V) conectada à entrada Micro-USB do ESP32. O microcontrolador utiliza seu regulador interno (LDO) para converter e distribuir 3,3V de forma segura para os sensores e o cartão SD. O consumo total estimado em pleno funcionamento (Wi-Fi ativo, módulo SD escrevendo e LEDs acesos) não ultrapassa 350mA. Esse valor é perfeitamente suportado pelo limite padrão de 500mA de uma porta USB 2.0 convencional, dispensando o uso de fontes externas durante o desenvolvimento e a apresentação.

7.4 Estrutura Física e Montagem

Para atender aos requisitos de robustez do projeto, o circuito final não utilizará protoboard. Todos os componentes serão organizados e soldados em uma placa perfurada ilhada ($5 \times 7\text{ cm}$). O microcontrolador será fixado por meio de barras de pinos fêmea, permitindo sua remoção para manutenção sem a necessidade de dessoldagem.

7.5 Mapa de Conexões (Pinagem Validada)

A distribuição das portas foi estruturada consultando o *datasheet* oficial do ESP32 DevKitC V4. Foram rigorosamente evitados pinos classificados como *Input Only* ou *Strapping Pins* (que afetam o boot do sistema) para os atuadores.

Tabela 1: Mapeamento Definitivo de Pinagem do Sistema

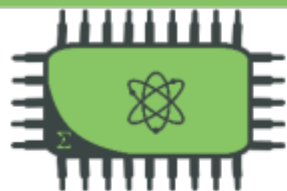
Componente	Função / Protocolo	Pino do ESP32
Sensor AHT20	I2C (SDA)	GPIO 21
Sensor AHT20	I2C (SCL)	GPIO 22
Módulo SD	SPI (CS / SS)	GPIO 5
Módulo SD	SPI (MOSI)	GPIO 23
Módulo SD	SPI (MISO)	GPIO 19
Módulo SD	SPI (SCK)	GPIO 18
LED Verde	Saída Digital (Ligado)	GPIO 27
LED Amarelo	Saída Digital (Automático)	GPIO 26
LED Vermelho	Saída Digital (Desligado)	GPIO 25

Brasília, 2 de maio de 2026.

ELETRONJUN – UNIVERSIDADE DE BRASÍLIA
EMPRESA JÚNIOR DE ENGENHARIA ELETRÔNICA

Área Especial de Indústria – Projeção A | 72444-240 | Gama/DF

www.eletronjun.com.br | contato@eletronjun.com.br



8 Descrição de Alto nível

8.1 Subsistema de Alimentação e Regulação

Este subsistema gerencia a conversão e distribuição de energia para garantir que cada componente opere dentro de sua faixa de tensão nominal, prevenindo danos ao hardware.

- **Entrada de Energia:** O sistema recebe alimentação de 5V via porta Micro-USB, conectada diretamente ao computador ou fonte externa.
- **Regulação de Tensão (LDO):** A placa DevKit utiliza um regulador de baixa queda (*Low-Dropout* - LDO) para converter os 5V da entrada em 3,3V estáveis, necessários para o funcionamento do chip ESP32 e do sensor AHT20.
- **Barramento de 5V:** Este barramento alimenta o módulo de cartão SD, que por sua vez possui regulação própria e *level shifters* internos para garantir a integridade dos sinais lógicos.

8.2 Subsistema de Sensoriamento e Armazenamento

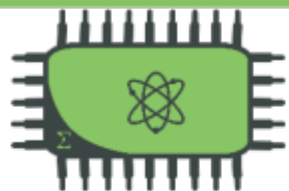
Responsável por ler as informações do ambiente e salvá-las fisicamente.

- **Sensor:** O AHT20 mede temperatura e umidade usando o protocolo I2C.
- **Gravação:** As leituras são salvas no cartão SD em arquivos .csv via protocolo SPI, garantindo que os dados fiquem guardados mesmo sem internet.

8.3 Subsistema de Interface e Atuação

Proporciona ao usuário uma visualização imediata do status operacional do dispositivo através de sinalizações luminosas.

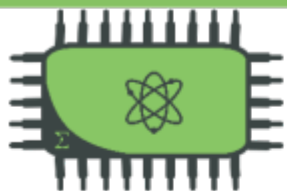
- **Sinalização Visual:** Três LEDs (Verde, Amarelo e Vermelho) compõem a interface física, permitindo identificar visualmente os estados de operação do hardware, estes leds funcionarão como: VERDE (ESTADO LIGADO), AMARELO (ESTADO AUTOMÁTICO), VERMELHO (ESTADO DESLIGADO)
- **Proteção Física:** Resistores limitadores de corrente são empregados em série com os LEDs, protegendo as saídas digitais do microcontrolador contra drenagem excessiva de corrente.



8.4 Subsistema de Comunicação e Arquitetura de Software

Este bloco gerencia a conectividade sem fio e a entrega da interface de usuário, permitindo o monitoramento remoto e a configuração do dispositivo em tempo real.

- **Conectividade Wi-Fi:** O microcontrolador opera nos modos *Station* (conectado a uma rede local) ou *Access Point* (criando sua própria rede para configuração inicial), utilizando a pilha TCP/IP nativa do ESP32.
- **Servidor Web Assíncrono:** Utiliza-se a biblioteca `ESPAsyncWebServer` para processar requisições HTTP de forma não bloqueante. Isso garante que a entrega das páginas do site não interrompa as tarefas críticas, como a leitura do sensor AHT20 ou a escrita de logs no cartão SD.
- **Gerenciamento de Arquivos (LittleFS):** Os arquivos de *front-end* (HTML, CSS e JavaScript) são armazenados em uma partição da memória flash do microcontrolador organizada pelo sistema de arquivos LittleFS. Essa abordagem separa a lógica de controle (C++) da interface visual, otimizando o uso da memória RAM.
- **Interatividade (JSON e AJAX):** A troca de dados entre o hardware e o navegador ocorre via pacotes JSON. Através de requisições assíncronas (AJAX), o site atualiza os valores de temperatura e umidade na tela automaticamente, eliminando a necessidade de recarregar a página pelo usuário.



9 Descrição de Firmware

O firmware será desenvolvido em C++ para o ESP32, utilizando uma lógica assíncrona. Isso permite que o processador cuide da leitura dos sensores e do controle dos LEDs ao mesmo tempo em que mantém o servidor web funcionando, sem que uma tarefa trave a outra. A estrutura principal será uma máquina de estados, que define como o hardware deve se comportar em cada modo (Desligado, Automático ou Ligado).

9.1 Sequência de Boot e Inicialização

Assim que o sistema é ligado, o firmware executa os preparativos básicos para operar:

- **Setup de Hardware:** O código inicia a comunicação I2C com o sensor AHT20 e a comunicação SPI com o módulo de cartão microSD
- **Conexão Wi-Fi:** O ESP32 busca a rede configurada e, após conectar, inicia o servidor web na porta 80 para permitir o acesso pela interface

9.2 Ciclo de Execução e Máquina de Estados

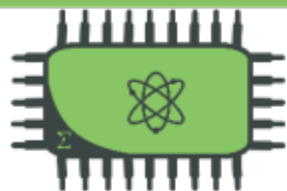
No laço principal do código, o firmware monitora as variáveis de estado e o tempo:

- **Seleção de Modo:** O sistema verifica qual modo está ativo. No "Automático", ele faz leituras periódicas; no "Ligado", fica pronto para comandos; e no "Desligado", interrompe as leituras e acende o LED vermelho
- **Lógica de Sensoriamento:** O firmware lê o sinal do sensor e o converte para valores reais de temperatura ($^{\circ}C$) e umidade (%)
- **Processamento de Alertas:** O código compara os dados com os limites definidos e, caso os valores saiam do normal, envia um sinal de alerta para a interface web

9.3 Gerenciamento de Arquivos no microSD

O sistema garante que as medições fiquem guardadas fisicamente:

- **Rotina de Datalogger:** A cada nova coleta, o firmware abre o arquivo `dados.csv` no modo de anexo, salva a nova linha de informação e fecha o arquivo logo em seguida para garantir a segurança dos dados
- **Envio do Histórico:** Quando solicitado via site, o firmware lê as informações salvas no cartão e as envia para serem exibidas na tela do usuário



9.4 Protocolo de Comunicação Web

Esta parte cuida da troca de informações entre o código e o navegador:

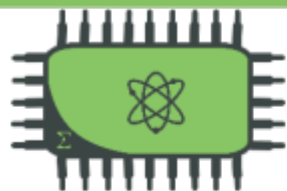
- **Tratamento de Requisições:** O servidor processa pedidos como a troca de modo (rota /modo) ou o envio dos dados atuais (rota /dados) em formato JSON
- **Sincronização de LEDs:** Sempre que o modo de operação muda pela interface, o firmware atualiza imediatamente o estado dos pinos GPIO, acendendo o LED correspondente (Verde, Amarelo ou Vermelho)

Brasília, 2 de maio de 2026.

ELETRONJUN – UNIVERSIDADE DE BRASÍLIA
EMPRESA JÚNIOR DE ENGENHARIA ELETRÔNICA

Área Especial de Indústria – Projeção A| 72444-240 | Gama/DF

www.eletronjun.com.br | contato@eletronjun.com.br



10 Bill of Materials

Descrição da lista de materiais a serem utilizados e seu respectivo preço...

ID	Componente	Descrição	Qntd	Preço Unit. (R\$)
1	ESP32 DevKitC V4	MCU Wi-Fi WROOM-32D	1	45,00
2	Sensor AHT20	Sensor Temp/Umid I2C	1	11,00
3	Módulo MicroSD	SPI com level shifter	1	9,90
4	Cartão microSD 8GB	Classe 10, FAT32	1	15,00
5	LED Verde	LED 5mm	1	0,50
6	LED Amarelo	LED 5mm	1	0,50
7	LED Vermelho	LED 5mm	1	0,50
8	Resistor 220Ω	1/4W ±5%	3	0,25
9	Header fêmea	Barra 2.54mm	1	3,00
10	Placa perfurada	PCB universal 5x7cm	1	8,00

Tabela 2: Bill of Materials